aZeroECS

Generated by Doxygen 1.9.3

1 Hierarchical Index 1
1.1 Class Hierarchy . . . 1

2 Class Index 3
2.1 Class List . . . 3

3 File Index 5
3.1 File List . . . 5

4 Class Documentation 7
4.1 aZeroECS::ComponentArray< T > Struct Template Reference . . . 7
4.1.1 Member Function Documentation . . . 7
4.1.1.1 Remove() . . . 7
4.2 aZeroECS::ComponentArrayBase Struct Reference . . . 8
4.2.1 Member Function Documentation . . . 8
4.2.1.1 Remove() . . . 8
4.3 aZeroECS::ComponentManager Class Reference . . . 9
4.3.1 Detailed Description . . . 9
4.3.2 Member Function Documentation . . . 10
4.3.2.1 AddComponent() [1/2] . . . 10
4.3.2.2 AddComponent() [2/2] . . . 10
4.3.2.3 GetComponent() [1/2] . . . 10
4.3.2.4 GetComponent() [2/2] . . . 11
4.3.2.5 GetComponentArray() . . . 11
4.3.2.6 GetComponentBit() . . . 12
4.3.2.7 GetComponentFast() [1/2] . . . 12
4.3.2.8 GetComponentFast() [2/2] . . . 12
4.3.2.9 HasComponent() . . . 13
4.3.2.10 RegisterComponent() . . . 13
4.3.2.11 RemoveComponent() [1/2] . . . 13
4.3.2.12 RemoveComponent() [2/2] . . . 14
4.4 aZeroECS::Entity Struct Reference . . . 14
4.4.1 Detailed Description . . . 15
4.5 aZeroECS::EntityManager Class Reference . . . 15
4.5.1 Detailed Description . . . 15
4.5.2 Member Function Documentation . . . 15
4.5.2.1 CreateEntity() . . . 15
4.5.2.2 RemoveEntity() . . . 15
4.6 MappedVector< T > Class Template Reference . . . 16
4.6.1 Detailed Description . . . 16
4.6.2 Constructor & Destructor Documentation . . . 16
4.6.2.1 MappedVector() . . . 16
4.6.3 Member Function Documentation . . . 17

4.6.3.1 Add() . . . 17
4.6.3.2 Contains() . . . 17
4.6.3.3 Expand() . . . 18
4.6.3.4 GetObjectByID() . . . 18
4.6.3.5 GetObjects() . . . 18
4.6.3.6 Remove() . . . 19
4.6.3.7 ShrinkToFit() . . . 19
4.7 aZeroECS::System Class Reference . . . 19
4.7.1 Detailed Description . . . 20
4.7.2 Member Function Documentation . . . 20
4.7.2.1 Bind() . . . 20
4.7.2.2 UnBind() . . . 20
4.7.2.3 Update() . . . 21
4.8 aZeroECS::SystemManager Class Reference . . . 21
4.8.1 Detailed Description . . . 21
4.8.2 Member Function Documentation . . . 21
4.8.2.1 EntityUpdated() . . . 21
4.8.2.2 RegisterSystem() . . . 22

5 File Documentation **23**
5.1 aZeroECS.h . . . 23
5.2 ComponentManager.h . . . 23
5.3 Entity.h . . . 25
5.4 EntityManager.h . . . 25
5.5 MappedVector.h . . . 26
5.6 System.h . . . 27
5.7 SystemManager.h . . . 27

Index **29**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

aZeroECS::ComponentArrayBase 8
  aZeroECS::ComponentArray< T > 7
aZeroECS::ComponentManager 9
aZeroECS::Entity 14
aZeroECS::EntityManager 15
MappedVector< T > 16
MappedVector< aZeroECS::Entity > 16
aZeroECS::System 19
aZeroECS::SystemManager 21

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

aZeroECS::ComponentArray< T > 7

aZeroECS::ComponentArrayBase 8

aZeroECS::ComponentManager
Used to store, manage, and register components 9

aZeroECS::Entity
Contains a unique ID and a bitmask containing which components the Entity currently has . . 14

aZeroECS::EntityManager
Creates and recycles Entity objects 15

MappedVector< T >
Stores objects inside a vector and uses a unique int ID to return a specific object. The vector will always be packed and can be iterated linearly 16

aZeroECS::System
An abstract base class which can be inherited and registered with SystemManager::RegisterSystem(). Each inheriting subclass should specify what set of components an Entity needs to have to be bound when System::Bind() is called. This can be done by writing "m_componentMask.set(m↩_componentManager.GetComponentBit<ComponentClassName>());" inside the subclass constructor. A component have to be registered using ComponentManager::RegisterComponent() prior to a System subclass constructor being called for ComponentManager::GetComponentBit() to work 19

aZeroECS::SystemManager
Registers System subclasses for aZeroECS 21

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

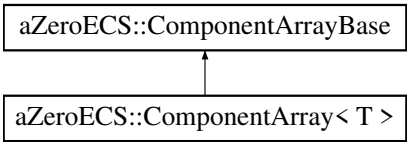
aZeroECS.h . . . . . ??
ComponentManager.h . . . . . ??
Entity.h . . . . . ??
EntityManager.h . . . . . ??
MappedVector.h . . . . . ??
System.h . . . . . ??
SystemManager.h . . . . . ??

Chapter 4

Class Documentation

4.1 aZeroECS::ComponentArray< T > Struct Template Reference

Inheritance diagram for aZeroECS::ComponentArray< T >:



Public Member Functions

- virtual void Remove (int id) override

Public Attributes

- MappedVector< T > **m_components**

4.1.1 Member Function Documentation

4.1.1.1 Remove()

```
template<typename T >
virtual void aZeroECS::ComponentArray< T >::Remove (
            int id )  [inline], [override], [virtual]
```

Removes the component matching the specified Entity ID.

Parameters

id	Unique Entity ID

Returns

void

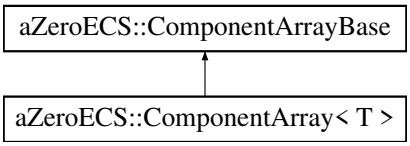
Implements aZeroECS::ComponentArrayBase.

The documentation for this struct was generated from the following file:

- ComponentManager.h

4.2 aZeroECS::ComponentArrayBase Struct Reference

Inheritance diagram for aZeroECS::ComponentArrayBase:



Public Member Functions

- virtual void Remove (int id)=0

4.2.1 Member Function Documentation

4.2.1.1 Remove()

```
virtual void aZeroECS::ComponentArrayBase::Remove (
            int id )  [pure virtual]
```

Pure virtual which is overriden by ComponentArray to allow MappedVector::Remove() without having to specify template parameters.

Parameters

id	Unique Entity ID

Returns

void

Implemented in aZeroECS::ComponentArray< T >.

The documentation for this struct was generated from the following file:

- ComponentManager.h

4.3 aZeroECS::ComponentManager Class Reference

Used to store, manage, and register components.

```
#include <ComponentManager.h>
```

Public Member Functions

- **ComponentManager** (SystemManager &systemManager)
- template<typename T >
  void RegisterComponent ()
- template<typename T >
  bool HasComponent (const Entity &entity) const
- template<typename T >
  void AddComponent (Entity &entity)
- template<typename T >
  void AddComponent (Entity &entity, const T &data)
- template<typename T >
  T * GetComponent (const Entity &entity)
- template<typename T >
  T * GetComponent (ComponentArray< T > &componentArray, const Entity &entity)
- template<typename T >
  T * GetComponentFast (const Entity &entity)
- template<typename T >
  T * GetComponentFast (ComponentArray< T > &componentArray, const Entity &entity)
- template<typename T >
  void RemoveComponent (Entity &entity)
- void RemoveComponent (Entity &entity, std::type_index typeIndex)
- template<typename T >
  ComponentArray< T > & GetComponentArray ()
- template<typename T >
  short GetComponentBit () const

Friends

- class **EntityManager**

4.3.1 Detailed Description

Used to store, manage, and register components.

4.3.2 Member Function Documentation

4.3.2.1 AddComponent() [1/2]

```
template<typename T >
void aZeroECS::ComponentManager::AddComponent (
            Entity & entity )  [inline]
```

Adds a component of type T to the input Entity.

Parameters

entity	The Entity to add the component to

Returns

void

4.3.2.2 AddComponent() [2/2]

```
template<typename T >
void aZeroECS::ComponentManager::AddComponent (
            Entity & entity,
            const T & data )  [inline]
```

Adds a component of type T to the input Entity. Initiates the new component with the input data.

Parameters

entity	The Entity to add the component to
data	Initial data for the component

Returns

void

4.3.2.3 GetComponent() [1/2]

```
template<typename T >
T * aZeroECS::ComponentManager::GetComponent (
            ComponentArray< T > & componentArray,
            const Entity & entity )  [inline]
```

Returns a pointer to the component of type T within the input ComponentArray<T> for the input Entity. This method returns nullptr if the input Entity doesn't have a component of type T. TODO: TRY TO AVOID GOING THROUGH THE MAP TO ACCESS THE COMPONENT ARRAY

Parameters

componentArray	The ComponentArray<T> to get the component from
entity	The Entity to get the component for

Returns

void

4.3.2.4 GetComponent() [2/2]

```
template<typename T >
T * aZeroECS::ComponentManager::GetComponent (
            const Entity & entity )  [inline]
```

Returns a pointer to the component of type T for the input Entity. Consider using ComponentManager::GetComponentArray<T>() in conjunctions with ComponentManager::GetComponent(ComponentArray<T>& componentArray, const Entity& entity) when accessing multiple components of the same type in a row. This will avoid the additional ComponentArray<T> lookup time which this method has. This method returns nullptr if the input Entity doesn't have a component of type T. TODO: TRY TO AVOID GOING THROUGH THE MAP TO ACCESS THE COMPONENT ARRAY

Parameters

entity	The Entity to get the component for

Returns

void

4.3.2.5 GetComponentArray()

```
template<typename T >
ComponentArray< T > & aZeroECS::ComponentManager::GetComponentArray ( )  [inline]
```

Returns a reference to a ComponentArray<T>.

Returns

ComponentArray<T>&

4.3.2.6 GetComponentBit()

```
template<typename T >
short aZeroECS::ComponentManager::GetComponentBit ( ) const  [inline]
```

Returns the std::bitset bit index for the component of type T.

Returns

short

4.3.2.7 GetComponentFast() [1/2]

```
template<typename T >
T * aZeroECS::ComponentManager::GetComponentFast (
            ComponentArray< T > & componentArray,
            const Entity & entity )  [inline]
```

Returns a pointer to the component of type T within the input ComponentArray<T> for the input Entity. This method will might crash if the Entity doesn't have a component of type T. This avoid unneccessary if-statements. Consider using ComponentManager::GetComponent() or atleast ComponentManager::HasComponent<T>() if you want to avoid this. TODO: TRY TO AVOID GOING THROUGH THE MAP TO ACCESS THE COMPONENT ARRAY

Parameters

componentArray	The ComponentArray<T> to get the component from
entity	The Entity to get the component for

Returns

void

4.3.2.8 GetComponentFast() [2/2]

```
template<typename T >
T * aZeroECS::ComponentManager::GetComponentFast (
            const Entity & entity )  [inline]
```

Returns a pointer to the component of type T for the input Entity. Consider using ComponentManager::GetComponentArray<T>() in conjunctions with ComponentManager::GetComponent(ComponentArray<T>& componentArray, const Entity& entity) when accessing multiple components of the same type in a row. This will avoid the additional ComponentArray<↩ T> lookup time which this method has. This method will might crash if the Entity doesn't have a component of type T. This avoid unneccessary if-statements. Consider using ComponentManager::GetComponent() or atleast ComponentManager::HasComponent<T>() if you want to avoid this. TODO: TRY TO AVOID GOING THROUGH THE MAP TO ACCESS THE COMPONENT ARRAY

Parameters

entity	The Entity to get the component for

Returns

void

4.3.2.9 HasComponent()

```
template<typename T >
bool aZeroECS::ComponentManager::HasComponent (
            const Entity & entity ) const  [inline]
```

Checks whether or not the input Entity currently has a component of type T.

Parameters

entity	The Entity to check

Returns

bool TRUE: The Entity has a component of type T, FALSE: The Entity doesn't have a component of type T

4.3.2.10 RegisterComponent()

```
template<typename T >
void aZeroECS::ComponentManager::RegisterComponent ( )  [inline]
```

Registers a new component type of type T for the ComponentManager. This should be called once for each component that the ComponentManager should support.

Returns

void

4.3.2.11 RemoveComponent() [1/2]

```
template<typename T >
void aZeroECS::ComponentManager::RemoveComponent (
            Entity & entity )  [inline]
```

Removes the component of type T for the input Entity.

Parameters

entity	The Entity to remove the component for

Returns

void

4.3.2.12 RemoveComponent() [2/2]

```
void aZeroECS::ComponentManager::RemoveComponent (
            Entity & entity,
            std::type_index typeIndex )  [inline]
```

Removes the component of type T matching the input std::type_index for the input Entity.

Parameters

entity	The Entity to remove the component for
typeIndex	The std::type_index which matches a registered type T component

Returns

void

The documentation for this class was generated from the following file:

- ComponentManager.h

4.4 aZeroECS::Entity Struct Reference

Contains a unique ID and a bitmask containing which components the Entity currently has.

```
#include <Entity.h>
```

Public Member Functions

- **Entity** (int id)

Public Attributes

- int **m_id** = -1
- std::bitset< MAXCOMPONENTS > **m_componentMask**

4.4.1 Detailed Description

Contains a unique ID and a bitmask containing which components the Entity currently has.

The documentation for this struct was generated from the following file:

- Entity.h

4.5 aZeroECS::EntityManager Class Reference

Creates and recycles Entity objects.

```
#include <EntityManager.h>
```

Public Member Functions

- **EntityManager** (ComponentManager &componentManager)
- Entity && CreateEntity ()
- void RemoveEntity (Entity &entity)

4.5.1 Detailed Description

Creates and recycles Entity objects.

4.5.2 Member Function Documentation

4.5.2.1 CreateEntity()

```
Entity && aZeroECS::EntityManager::CreateEntity ( )  [inline]
```

Creates a new Entity object with a unique ID.

Returns

Entity&&

4.5.2.2 RemoveEntity()

```
void aZeroECS::EntityManager::RemoveEntity (
            Entity & entity )  [inline]
```

Recycles the input Entity so that it's unique ID can be reused when calling EntityManager::CreateEntity(). Removes all components for the Entity and unbinds it from all registered System subclasses. Changes the input Entity object ID to -1 to indicate that it is recycled. Doesn't do anything if the input Entity ID is -1.

Parameters

The	Entity which should be recycled

Returns

void

The documentation for this class was generated from the following file:

- EntityManager.h

4.6 MappedVector< T > Class Template Reference

Stores objects inside a vector and uses a unique int ID to return a specific object. The vector will always be packed and can be iterated linearly.

```
#include <MappedVector.h>
```

Public Member Functions

- MappedVector (int startSize=100)
- void Add (int id, const T &value)
- void Remove (int id)
- void ShrinkToFit ()
- void Expand (int numElements)
- bool Contains (int id) const
- T * GetObjectByID (int id)
- std::vector< T > & GetObjects ()

4.6.1 Detailed Description

template<typename T>
class MappedVector< T >

Stores objects inside a vector and uses a unique int ID to return a specific object. The vector will always be packed and can be iterated linearly.

4.6.2 Constructor & Destructor Documentation

4.6.2.1 MappedVector()

```
template<typename T >
MappedVector< T >::MappedVector (
            int startSize = 100 )  [inline]
```

Allocates a specified amount of memory for the vector.

Parameters

startSize	Number of elements that the vector should allocate capacity for

4.6.3 Member Function Documentation

4.6.3.1 Add()

```
template<typename T >
void MappedVector< T >::Add (
            int id,
            const T & value )  [inline]
```

Stores the input value inside the vector and makes it accessible through the input id.

Parameters

id	Unique id to store the value for
value	Initial value for the object

Returns

void

4.6.3.2 Contains()

```
template<typename T >
bool MappedVector< T >::Contains (
            int id ) const  [inline]
```

Checks whether or not an object is stored for the input id.

Parameters

id	Unique id which should be checked

Returns

bool TRUE: An object with the input id exists, FALSE: No object with the input id exists

4.6.3.3 Expand()

```
template<typename T >
void MappedVector< T >::Expand (
            int numElements )  [inline]
```

Expands the vectors memory.

Parameters

numElements	Number of elements to expand the memory with

Returns

void

4.6.3.4 GetObjectByID()

```
template<typename T >
T * MappedVector< T >::GetObjectByID (
            int id )  [inline]
```

Returns a pointer to the element for the input id.

Parameters

id	The id to retrieve the element with.

Returns

T*

4.6.3.5 GetObjects()

```
template<typename T >
std::vector< T > & MappedVector< T >::GetObjects ( )  [inline]
```

Returns a reference to the vector. To iterate over it in a safe manner, use structure binding with an iterator or loop using the std::vector::size().

Returns

std::vector<T>&

4.6.3.6 Remove()

```
template<typename T >
void MappedVector< T >::Remove (
            int id )  [inline]
```

Free's up the input id from the MappedVector and removes the object inside the vector.

Parameters

id	Unique id which should have it's value removed

Returns

void

4.6.3.7 ShrinkToFit()

```
template<typename T >
void MappedVector< T >::ShrinkToFit ( )  [inline]
```

Shrinks the vector to fit perfectly around the current number of elements.

Returns

void

The documentation for this class was generated from the following file:

- MappedVector.h

4.7 aZeroECS::System Class Reference

An abstract base class which can be inherited and registered with SystemManager::RegisterSystem(). Each inheriting subclass should specify what set of components an Entity needs to have to be bound when System::Bind() is called. This can be done by writing "m_componentMask.set(m_componentManager.Get↩ ComponentBit<ComponentClassName>());" inside the subclass constructor. A component have to be registered using ComponentManager::RegisterComponent() prior to a System subclass constructor being called for ComponentManager::GetComponentBit() to work.

```
#include <System.h>
```

Public Member Functions

- **System** (ComponentManager &componentManager)
- virtual bool Bind (const Entity &entity)
- virtual bool UnBind (const Entity &entity)
- virtual void Update ()=0

Protected Attributes

- std::bitset< MAXCOMPONENTS > **m_componentMask**
- ComponentManager & **m_componentManager**
- MappedVector< Entity > **m_entities**

Friends

- class **SystemManager**

4.7.1 Detailed Description

An abstract base class which can be inherited and registered with SystemManager::RegisterSystem(). Each inheriting subclass should specify what set of components an Entity needs to have to be bound when System::Bind() is called. This can be done by writing "m_componentMask.set(m_componentManager.Get↩ ComponentBit<ComponentClassName>());" inside the subclass constructor. A component have to be registered using ComponentManager::RegisterComponent() prior to a System subclass constructor being called for ComponentManager::GetComponentBit() to work.

4.7.2 Member Function Documentation

4.7.2.1 Bind()

```
virtual bool aZeroECS::System::Bind (
            const Entity & entity )  [inline], [virtual]
```

Binds the input Entity if it's m_componentMask matches the m_componentMask set in the System subclass constructor.

Parameters

entity	The Entity which should be bound

Returns

bool TRUE: Entity is either already bound or bound during this call of System::Bind(). FALSE: Entity::m_↩ componentMask isn't compatible with System::m_componentMask

4.7.2.2 UnBind()

```
virtual bool aZeroECS::System::UnBind (
            const Entity & entity )  [inline], [virtual]
```

Unbinds the input Entity.

Parameters

entity	The Entity which should be unbound

Returns

bool TRUE: Entity was unbound. FALSE: Entity wasn't bound

4.7.2.3 Update()

```
virtual void aZeroECS::System::Update ( )  [pure virtual]
```

Pure virtual method which should be overridden by the inheriting subclass.

Returns

void

The documentation for this class was generated from the following file:

- System.h

4.8 aZeroECS::SystemManager Class Reference

Registers System subclasses for aZeroECS.

```
#include <SystemManager.h>
```

Public Member Functions

- template<typename T , typename... Args>
  std::shared_ptr< T > RegisterSystem (ComponentManager &componentManager, Args &&...args)
- void EntityUpdated (const Entity &entity)

Friends

- class **ComponentManager**

4.8.1 Detailed Description

Registers System subclasses for aZeroECS.

4.8.2 Member Function Documentation

4.8.2.1 EntityUpdated()

```
void aZeroECS::SystemManager::EntityUpdated (
            const Entity & entity )  [inline]
```

Checks if the Entity should be bound or unbound from a registered System subclass depending on it's component bitmask. Binds the Entity to a registered System subclass if it has the neccessary components. Unbinds the Entity from a registered System subclass if it doesn't have the neccessary components.

Parameters

entity	The Entity which should be checked

Returns

void

4.8.2.2 RegisterSystem()

```
template<typename T , typename...  Args>
std::shared_ptr< T > aZeroECS::SystemManager::RegisterSystem (
            ComponentManager & componentManager,
            Args &&...  args )  [inline]
```

Registers the input System subclass and returns a std::shared_ptr to the shared memory block.

Parameters

componentManager	The ComponentManager which the System subclass can use to access components
args	An arbitrary number of arguments which will be passed to the constructor of System subclass after componentManager

Returns

std::shared_ptr<T>

The documentation for this class was generated from the following file:

- SystemManager.h

Chapter 5

File Documentation

5.1 aZeroECS.h

```
#pragma once
#include "EntityManager.h"
```

5.2 ComponentManager.h

```
#pragma once
#include <unordered_map>
#include <typeindex>
#include <memory>
#include "SystemManager.h"

namespace aZeroECS
{
    struct ComponentArrayBase
    {
        ComponentArrayBase() = default;
        virtual ~ComponentArrayBase() = default;

        virtual void Remove(int id) = 0;
    };

    template<typename T>
    struct ComponentArray :  public ComponentArrayBase
    {
        ComponentArray() = default;

        MappedVector<T> m_components;

        virtual void Remove(int id)override
{
            m_components.Remove(id);
        }
    };

    class ComponentManager
    {
        friend class EntityManager;

    private:
        std::unordered_map<std::type_index, std::unique_ptr<ComponentArrayBase» m_componentArrayMap;
        std::unordered_map<std::type_index, short> m_typeToBitflag;
        SystemManager& m_systemManager;

    public:
        ComponentManager(SystemManager& systemManager)
            :m_systemManager(systemManager) { }

        template<typename T>
        void RegisterComponent()
        {
            std::type_index typeIndex = std::type_index(typeid(T));

            m_typeToBitflag.emplace(typeIndex, m_componentArrayMap.size());
            m_componentArrayMap.emplace(typeIndex, std::make_unique<ComponentArray<T>>());
```

```
        }

        template<typename T>
        bool HasComponent(const Entity& entity)const
{
            return entity.m_componentMask.test(static_cast<size_t>(m_typeToBitflag.at(std::type_index(typeid(T)))));
        }

        template<typename T>
        void AddComponent(Entity& entity)
        {
            entity.m_componentMask.set(static_cast<size_t>(m_typeToBitflag.at(std::type_index(typeid(T)))), true);

            ComponentArray<T>* componentArray = static_cast<ComponentArray<T>*>(m_componentArrayMap.at(std::type_index(typeid(T))).get());

            componentArray->m_components.Add(entity.m_id, std::move(T()));

            m_systemManager.EntityUpdated(entity);

        }

        template<typename T>
        void AddComponent(Entity& entity, const T& data)
        {
            entity.m_componentMask.set(static_cast<size_t>(m_typeToBitflag.at(std::type_index(typeid(T)))), true);

            ComponentArray<T>* componentArray = static_cast<ComponentArray<T>*>(m_componentArrayMap.at(std::type_index(typeid(T))).get());

            componentArray->m_components.Add(entity.m_id, data);

            m_systemManager.EntityUpdated(entity);

        }

        template<typename T>
        T* GetComponent(const Entity& entity)
        {
            ComponentArray<T>* componentArray = static_cast<ComponentArray<T>*>(m_componentArrayMap.at(std::type_index(typeid(T))).get());
            if(componentArray->m_components.Contains(entity.m_id))
                return componentArray->m_components.GetObjectByID(entity.m_id);

            return nullptr;
        }

        template<typename T>
        T* GetComponent(ComponentArray<T>& componentArray, const Entity& entity)
        {
            if (componentArray.m_components.Contains(entity.m_id))
                return componentArray.m_components.GetObjectByID(entity.m_id);

            return nullptr;
        }

        template<typename T>
        T* GetComponentFast(const Entity& entity)
        {
            ComponentArray<T>* componentArray = static_cast<ComponentArray<T>*>(m_componentArrayMap.at(std::type_index(typeid(T))).get());
            return componentArray->m_components.GetObjectByID(entity.m_id);
        }

        template<typename T>
        T* GetComponentFast(ComponentArray<T>& componentArray, const Entity& entity)
        {
            return componentArray.m_components.GetObjectByID(entity.m_id);
        }

        template<typename T>
        void RemoveComponent(Entity& entity)
        {
            entity.m_componentMask.set(static_cast<size_t>(m_typeToBitflag.at(std::type_index(typeid(T)))), false);

            ComponentArrayBase* base = m_componentArrayMap.at(std::type_index(typeid(T))).get();
            base->Remove(entity.m_id);

            m_systemManager.EntityUpdated(entity);
        }

        void RemoveComponent(Entity& entity, std::type_index typeIndex)
```

```
        {
            entity.m_componentMask.set(static_cast<size_t>(m_typeToBitflag.at(typeIndex)), false);

            ComponentArrayBase* base = m_componentArrayMap.at(typeIndex).get();
            base->Remove(entity.m_id);

            m_systemManager.EntityUpdated(entity);
        }

        template<typename T>
        ComponentArray<T>& GetComponentArray()
        {
            return
    *static_cast<ComponentArray<T>*>(m_componentArrayMap.at(std::type_index(typeid(T))).get());
        }

        template<typename T>
        short GetComponentBit()const
{
            return m_typeToBitflag.at(std::type_index(typeid(T)));
        }
    };
}
```

5.3 Entity.h

```
#pragma once
#include <bitset>

namespace aZeroECS
{
    const int MAXCOMPONENTS = 32;

    struct Entity
    {
        Entity(int id)
            :m_id(id) { }

        int m_id = -1;
        std::bitset<MAXCOMPONENTS> m_componentMask;
    };
}
```

5.4 EntityManager.h

```
#pragma once
#include <queue>
#include "Entity.h"
#include "ComponentManager.h"

namespace aZeroECS
{
    class EntityManager
    {
    private:
        int m_currentMax = 0;
        int m_maxEntities = 0;
        std::queue<int> m_freeEntityIDs;
        ComponentManager& m_componentManager;

    public:
        EntityManager(ComponentManager& componentManager)
            :m_componentManager(componentManager) { }

        Entity&& CreateEntity()
        {
            if (!m_freeEntityIDs.empty())
            {
                int id = m_freeEntityIDs.front();
                m_freeEntityIDs.pop();
                return Entity(id);
            }

            int id = m_currentMax;
            m_currentMax++;
            return Entity(id);
        }

        void RemoveEntity(Entity& entity)
```

```
        {
            if (entity.m_id == -1)
                return;

            for (auto [index, bitFlag] :  m_componentManager.m_typeToBitflag)
            {
                if (entity.m_componentMask.test(bitFlag))
                {
                    m_componentManager.RemoveComponent(entity, index);
                }
            }

            m_componentManager.m_systemManager.EntityUpdated(entity);

            m_freeEntityIDs.push(entity.m_id);
            entity.m_id = -1;
        }
    };
}
```

5.5 MappedVector.h

```
#pragma once
#include <vector>
#include <unordered_map>

template<typename T>
class MappedVector
{
private:
    std::unordered_map<int, int> m_idToIndex;
    std::unordered_map<int, int> m_indexToId;
    std::vector<T> m_objects;

public:

    MappedVector(int startSize = 100)
    {
        m_objects.reserve(startSize);
    }

    void Add(int id, const T& value)
    {
        int index = m_objects.size();
        m_objects.emplace_back(value);
        m_idToIndex.emplace(id, index);
        m_indexToId[index] = id;
    }

    void Remove(int id)
    {
        int index = m_idToIndex.at(id);
        m_idToIndex.erase(id);

        int indexOfLast = m_objects.size() - 1;

        if (indexOfLast == index)
        {
            m_indexToId.erase(index);
            m_objects.erase(m_objects.begin() + index);
            return;
        }

        m_objects[index] = std::move(m_objects[indexOfLast]);

        int idOfLast = m_indexToId.at(indexOfLast);
        m_idToIndex.at(idOfLast) = index;

        m_indexToId.at(index) = idOfLast;
        m_indexToId.erase(indexOfLast);
        m_objects.erase(m_objects.begin() + indexOfLast);
    }

    void ShrinkToFit()
    {
        m_objects.shrink_to_fit();
    }

    void Expand(int numElements)
    {
        m_objects.reserve(m_objects.size() + numElements);
    }

```

```
    bool Contains(int id)const
{
        return m_idToIndex.count(id) > 0;
    }

    T* GetObjectByID(int id) { return &m_objects[m_idToIndex.at(id)]; }

     std::vector<T>& GetObjects() { return m_objects; }
};
```

5.6 System.h

```
#pragma once
#include "Entity.h"
#include "MappedVector.h"

namespace aZeroECS
{
    class ComponentManager;

    class System
    {
        friend class SystemManager;

    protected:
        std::bitset<MAXCOMPONENTS> m_componentMask;
        ComponentManager& m_componentManager;

        MappedVector<Entity> m_entities;
    public:
        System(ComponentManager& componentManager)
            :m_componentManager(componentManager) { }

        virtual bool Bind(const Entity& entity)
        {
            for (int i = 0; i < MAXCOMPONENTS; ++i)
            {
                if (m_componentMask.test(i))
                {
                    if (!entity.m_componentMask.test(i))
                    {
                        return false;
                    }
                }
            }

            if (m_entities.Contains(entity.m_id))
                return true;

            m_entities.Add(entity.m_id, entity);

            return true;
        }

        virtual bool UnBind(const Entity& entity)
        {
            if (!m_entities.Contains(entity.m_id))
                return false;

            m_entities.Remove(entity.m_id);

            return true;
        }

        virtual void Update() = 0;
    };
}
```

5.7 SystemManager.h

```
#pragma once
#include "System.h"

namespace aZeroECS
{
    class SystemManager
    {
        friend class ComponentManager;

```

```
    private:
        std::unordered_map<std::type_index, std::shared_ptr<System» m_registeredSystems;

    public:
        SystemManager() = default;

        template<typename T, typename...Args>
        std::shared_ptr<T> RegisterSystem(ComponentManager& componentManager, Args&&...args)
        {
            std::shared_ptr<T> newSystem = std::make_shared<T>(componentManager, args...);

            m_registeredSystems.emplace(std::type_index(typeid(T)), newSystem);

            return newSystem;
        }

        void EntityUpdated(const Entity& entity)
        {
            for (auto& system :  m_registeredSystems)
            {
                bool bound = system.second->Bind(entity);
                if (!bound)
                {
                    system.second->UnBind(entity);
                }
            }
        }
    };
}
```

Index

Add
MappedVector< T >, 17
AddComponent
aZeroECS::ComponentManager, 10
aZeroECS::ComponentArray< T >, 7
Remove, 7
aZeroECS::ComponentArrayBase, 8
Remove, 8
aZeroECS::ComponentManager, 9
AddComponent, 10
GetComponent, 10, 11
GetComponentArray, 11
GetComponentBit, 11
GetComponentFast, 12
HasComponent, 13
RegisterComponent, 13
RemoveComponent, 13, 14
aZeroECS::Entity, 14
aZeroECS::EntityManager, 15
CreateEntity, 15
RemoveEntity, 15
aZeroECS::System, 19
Bind, 20
UnBind, 20
Update, 21
aZeroECS::SystemManager, 21
EntityUpdated, 21
RegisterSystem, 22

Bind
aZeroECS::System, 20

Contains
MappedVector< T >, 17
CreateEntity
aZeroECS::EntityManager, 15

EntityUpdated
aZeroECS::SystemManager, 21
Expand
MappedVector< T >, 17

GetComponent
aZeroECS::ComponentManager, 10, 11
GetComponentArray
aZeroECS::ComponentManager, 11
GetComponentBit
aZeroECS::ComponentManager, 11
GetComponentFast
aZeroECS::ComponentManager, 12
GetObjectByID
MappedVector< T >, 18
GetObjects
MappedVector< T >, 18

HasComponent
aZeroECS::ComponentManager, 13

MappedVector
MappedVector< T >, 16
MappedVector< T >, 16
Add, 17
Contains, 17
Expand, 17
GetObjectByID, 18
GetObjects, 18
MappedVector, 16
Remove, 18
ShrinkToFit, 19

RegisterComponent
aZeroECS::ComponentManager, 13
RegisterSystem
aZeroECS::SystemManager, 22
Remove
aZeroECS::ComponentArray< T >, 7
aZeroECS::ComponentArrayBase, 8
MappedVector< T >, 18
RemoveComponent
aZeroECS::ComponentManager, 13, 14
RemoveEntity
aZeroECS::EntityManager, 15

ShrinkToFit
MappedVector< T >, 19

UnBind
aZeroECS::System, 20
Update
aZeroECS::System, 21